PUBLISHED Mar 23, 2026
ARTICLE UWTN 2026-002
VERSION 1.0.0
LICENCE CC-BY-4.0
ARCHIVED 2026-08-10T04:19:43Z
SOURCE https://www.underworldcode.org/our-journey-from-underworld2-to-underworld3/

Our Journey from Underworld2 to Underworld3

Louis Moresi[1], **John Mansour**[2], **Julian Giordani**[3], and **Romain Beucher**[1]

[1]Australian National University, [2]Monash University, [3]University of Sydney

DOI https://doi.org/10.6084/m9.figshare.33193566

Abstract

Underworld is a code for geodynamics — mantle convection, lithospheric deformation, subduction, ice flow. We solve coupled, nonlinear PDEs with complex rheologies using Lagrangian particles to track material history. The project has been running for 20 years: why did we start again ?

Keywords Underworld Code, Geodynamics, Tricks of the Trade

1. UW1 and UW2: Same Wolf, different clothes

Underworld1 assembled simulations from modular C components via XML configuration files: deterministic, reproducible, and rigid. Changing the physics meant writing C code and registering it in the XML framework. The target audience narrowed to people who could write C in a specific component architecture and, you'd have to admit, this is not a recipe for widespread adoption.

Underworld2 wrapped that C engine in Python. The original motivation was interoperability with the scientific Python ecosystem, but Python brought something we had not anticipated. The `Function` class gave users a composable interface for describing mathematics: build a viscosity from temperature dependence, yielding criteria, and material properties, all in Python. The Jupyter notebook became the natural home for model development. Romain Beucher's UWGeodynamics module showed that a structured geodynamics interface could sit comfortably on top of the core API.

The Function concept was the seed of a great idea. But underneath the Python layer, UW2 ran the same StGermain engine, the same finite element assembly routines, the same structured hex meshes. Python made Underworld usable by a much wider community. It did not make the engine extensible.

In some ways, UW2 delayed the inevitable. It was successful enough that the case for a full rewrite was hard to make. The Python layer papered over the cracks in the C engine, and because models ran and papers got published, the accumulated friction took years to become unbearable.

2. Why We Finally Started Again

The pressures that drove the rewrite existed in UW1. UW2 inherited all of them.

PETSc outgrew us. We started using PETSc when it was quite immature for our purposes. We built our own finite element engine on top — our own assembly, our own solvers, our own mesh management. Although we could see that PETSc had DMPlex for unstructured meshes, SNES for nonlinear solvers, and the pointwise function interface for weak-form assembly, we could not use any of that. Our architecture sat *beside* PETSc rather than on top of it. The inability to leverage SNES was particularly painful — nonlinear problems are the bread and butter of geodynamics, and we had locked ourselves out of PETSc's Newton solver framework.

StGermain. XML component loading, C object lifecycle management, dlopen gymnastics. Error messages from the C layer were cryptic. The `_function.py` source devoted 70 lines to producing useful error reports when something went wrong. UW3 was the opportunity to set all of this straight.

Hex meshes only. UW2 could only build structured hexahedral meshes with regular shapes. We wanted spherical shells for mantle convection, ellipsoidal geometry for regional models. We tried to make spherical meshes work within UW2 (and we certainly tried hard) but the mesh infrastructure was too tightly coupled to the Cartesian assumption. DMPlex gave us unstructured meshes, gmsh import, cubed-sphere construction, and adaptive refinement.

The build system. StGermain depended on PETSc, which depended on MPI and HDF5. SWIG generated Python bindings — thousands of `.py` files. On HPC filesystems, the sheer number of small files meant that `import underworld` would sometimes randomly fail. Docker became the recommended installation path, which solved the build problem but created others. The whole situation consumed far too much of the team's energy.

3. Functions v2: SymPy Shall Provide

We chose SymPy as UW3′s expression language for practical reasons: we needed a mature symbolic algebra library in Python that could represent PDEs. We got much more than we thought we would:-

Introspection. Every constitutive model, every boundary condition, every time derivative in UW3 is a SymPy expression. You can ask the code to show you the mathematics at any point. In a Jupyter notebook, the solver renders the weak form it assembled, the Jacobian it computed, the constitutive law after simplification. This turns out to be extraordinarily useful for debugging and for teaching — you can see what the solver actually solves, not what you hope it solves.

Lazy evaluation. Expressions build up symbolically, but nothing evaluates until the solver needs concrete numbers. Derivatives for Newton iteration are computed symbolically and deferred until compilation. This started as an implementation detail — a natural consequence of choosing SymPy — but it became the architectural keystone. Lazy evaluation enables *symbolic problem templates*: the Stokes solver, the Navier-Stokes solver, the Darcy solver are all defined as *equation* templates where the user fills in constitutive laws and the framework handles weak-form assembly, Jacobian derivation, and C code generation automatically.

Automatic Jacobians. Because a constitutive model is a symbolic expression, the framework differentiates it exactly. No hand-coded Jacobian contributions. No finite-difference approximations. This is what finally unlocked PETSc's SNES for us — the symbolic layer provides exactly the derivatives that Newton's method needs. And this does not just apply to Newton methods. If you want to solve adjoint problems, being able to create symbolic derivatives of the entire strong form is essential.

UW2's Function class was the right idea but we did not have sufficient depth in our implementation to cover arbitrarily complex operations. Functions were opaque C objects: you could compose them but not inspect, simplify, or differentiate them. Moving that concept into SymPy made it transparent. What started as a convenient expression language became the way the code thinks about physics. That evolution was not planned, but recognising it and embracing it shaped the best parts of UW3's design.

4. What UW3 Can Do That UW2 Could Not

The architectural changes are not abstract — they unlock concrete capabilities that we needed for years and could not build.

Boundary conditions on curved surfaces. PETSc's native boundary condition machinery assumes you can enumerate DOFs on a flat boundary and set their values directly. That breaks down on curved surfaces — a no-slip condition on a spherical shell, a free-slip condition on an irregular interface. In UW2, we had no good answer for this. UW3 uses a penalty approach: boundary conditions are expressed as additional terms in the weak form, weighted by a large penalty parameter. This is more flexible than it sounds. The same framework handles seepage boundary conditions (where fluid can leave but not enter a surface), frictional boundaries, and slip conditions on curved faults. These are not edge cases in geodynamics — they are everyday requirements that UW2 could not meet cleanly.

Coordinates without assumptions. UW2 was hardwired Cartesian. Every differential operator assumed x, y, z on a flat grid. UW3 defines coordinate systems through SymPy, and the differential operators — gradient, divergence, curl — auto-adjust for the geometry. Write your governing equations once; run them in Cartesian, cylindrical, or spherical coordinates without changing the physics. The same Stokes solver template works for a rectangular box and a spherical shell. The coordinate system supplies the metric tensors and Jacobians; the user supplies the constitutive law. This is what makes spherical and geographic meshes practical, not just possible.

Constitutive models without C. In UW2, adding a new rheology meant writing C code in the StGermain framework, compiling it, and wrapping it in Python. The barrier was high enough that most users never tried. In UW3, a constitutive model is a Python class with SymPy expressions for the stress-strain relationship. A domain scientist can write a new rheology — transverse isotropy, pressure-dependent plasticity, composite diffusion-dislocation creep — and test it in a notebook in an afternoon. The framework differentiates it for the Jacobian automatically. This is a direct consequence of the SymPy choice, but it is one capability that users find extremely helpful.

Time derivatives as swappable objects. UW2 had one time integration approach: explicit particle advection with second-order Runge-Kutta. UW3 treats the time derivative as a symbolic object with multiple implementations — Lagrangian (particles follow the flow), Semi-Lagrangian (unconditionally stable characteristic tracing), and Eulerian (fixed grid with advection correction). All are symbolic, all participate in the weak form, and you can swap between them without rewriting the solver. The schemes support variable-order BDF and Adams-Moulton integration with automatic order ramping for stability at startup. Changing the time discretisation of a problem is a one-line change, not a restructuring of the code.

5. What's Not Finished

Not everything is better yet. The particle machinery in UW3 is still catching up to UW2 in some respects. Population control — maintaining a good particle distribution as the mesh deforms and particles cluster or

deplete — is not yet as robust. The move from integer-keyed material mapping (`fn.branching.map` in UW2, which was beautifully simple) to level-set weighted composite representations is more general but less intuitive. There is work to do, and these are areas where contributions are welcome.

6. Where We Are

Underworld3 3.0.0 marks the point where the new architecture is mature enough to replace UW2 as the production tool. The symbolic pipeline is solid. The solver framework leverages modern PETSc properly. The mesh infrastructure handles the geometries that geodynamics actually needs. And the Python interface — the part of UW2 that actually worked — is better than ever, because SymPy makes it legible all the way down.

In upcoming posts, we go deeper into the machinery: how SymPy expressions become C code, how particles navigate a parallel mesh, how the units system tracks physical dimensions through the pipeline, and how we build geographic meshes for regional models.

The Underworld project is supported by AuScope and the Australian Government through the National Collaborative Research Infrastructure Strategy (NCRIS). Source code: github.com/underworldcode/underworld3